

EPPS6323 Knowledge Mining

Assignment 5

Name: Ann Nibana S L

1. NLP for text classification and prediction
 - a. Run nlp0x.R

Here we use nlp02.R script for text prediction.

```
## NLP 2: text prediction
## Purpose:
# Install required packages if not already installed
#required_packages <- c("tidyverse", "tidymodels", "textrecipes", "ranger", "workflows")
#new_packages <- required_packages[!(required_packages %in% installed.packages()[,"Package"])]
#if(length(new_packages)) install.packages(new_packages)
# Load libraries
library(tidyverse)
library(tidymodels)
library(textrecipes)
library(workflows)

# 1. Data Ingestion and Preparation
data200 <- read_csv("https://raw.githubusercontent.com/datagenetion/knowledgemining/refs/heads/master/data/km_sample_corpus_200.csv")
data200 <- data200 %>% mutate(label = factor(label))

set.seed(123) # For reproducibility
split <- initial_split(data200, prop = 0.7, strata = label)
train_data <- training(split)
test_data <- testing(split)

# 2. Define a Preprocessing Recipe
rec <- recipe(label ~ text, data = train_data) %>%
  step_tokenize(text) %>% # Tokenize the text
  step_stopwords(text) %>% # Remove stopwords
  step_tokenfilter(text, max_tokens = 100) %>% # Keep top 100 tokens
  step_tfidf(text) %>% # Convert tokens to TF-IDF features
#class(rec), it will give recipe the generalized approach

# 3. Specify a Random Forest Model with Tunable Hyperparameters
# We'll tune mtry (number of predictors sampled for splitting)
# and min_n (minimum number of observations in a node).
rf_spec <- rand_forest(
  trees = 100, # We'll keep trees fixed for this tuning example# more trees more stable, hierarchial clust
  mtry = tune(), # Number of predictors to sample at each split#set_engine("ranger")%>%pipe good for tree based models
  min_n = tune() # Minimum number of data points in a nodeset
) %>%
  set_engine("ranger") %>%
  set_mode("classification")#set_engine("ranger")%>%pipe good for tree based models

# 5. Set Up Cross-Validation
set.seed(123)
cv_folds <- vfold_cv(train_data, v = 5, strata = label)

# 6. Define a Grid for Hyperparameter Tuning
# Here, we specify a grid for mtry and min_n.
rf_grid <- grid_regular(
  mtry(range = c(5, 20)),
  min_n(range = c(2, 10)),
  levels = 5 # 5 levels for each hyperparameter
)
```

The dataset is read from a remote CSV file and contains short text documents labeled with topics such as culture, education etc . The labels are converted into factors to suit classification. To prepare for training and evaluation, the data is split into a **70% training set and 30% test set**, using `initial_split()` with stratification to avoid class imbalance during training.

Further Tokenization, Stopword removal filters keeping only the top 100 most frequent or informative words. TF-IDF conversion which filters tokens into numeric scores that reflect how important each word is to a given document compared to the entire corpus is done.

A **random forest model** defined with tunable hyperparameters is used here. The number of trees is fixed at 100.

```
# 7. Tune the Model Using Cross-Validation
set.seed(123)
tune_results <- tune_grid(
  wf,
  resamples = cv_folds(),
  grid = rf_grid,
  metrics = metric_set(accuracy, kap)
)

# Collect the best parameters based on accuracy
best_params <- select_best(tune_results, metric = "accuracy")
print(best_params)

# 8. Finalize the Workflow with the Best Hyperparameters
final_wf <- finalize_workflow(wf, best_params)

# Fit the final model on the full training data
final_fit <- final_wf %>% workflows::fit(data = train_data)

# 9. Evaluate the Final Model on the Test Set
final_preds <- predict(final_fit, new_data = test_data) %>%
  bind_cols(test_data)

# Performance Metrics
final_preds <- final_preds %>% mutate(label = as.factor(label))
final_metrics <- metric_set(accuracy, kap)(final_preds, truth = label, estimate = .pred_class)

print(final_metrics)

# Confusion Matrix
final_conf_mat <- conf_mat(final_preds, truth = label, estimate = .pred_class)
print(final_conf_mat)
```

Defines a regular tuning grid with `mtry` (number of predictors randomly sampled at each split) ranging from **5 to 20** and `min_n` (minimum number of data points in a node) ranging from **2 to 10**.

```
# 4. Create a Workflow Combining the Recipe and the Model Specification
wf <- workflow() %>%
  add_recipe(rec) %>%
  add_model(rf_spec)

# 5. Set Up Cross-Validation
set.seed(123)
cv_folds <- vfold_cv(train_data, v = 5, strata = label)

# 6. Define a Grid for Hyperparameter Tuning
# Here, we specify a grid for mtry and min_n.
rf_grid <- grid_regular(
  mtry(range = c(5, 20)),
  min_n(range = c(2, 10)),
  levels = 5 # 5 levels for each hyperparameter
)
```

```
>
> print(final_metrics)
# A tibble: 2 x 3
  .metric .estimator .estimate
  <chr>   <chr>      <dbl>
1 accuracy multiclass    0.7
2 kap      multiclass    0.667
> # Confusion Matrix
> final_conf_mat <- conf_mat(final_preds, truth = label, estimate = .pred_class)
> print(final_conf_mat)
```

	Truth									
Prediction	Culture	Education	Entertainment	Environment	Finance	Health	Politics	Sports	Technology	Travel
Culture	2	0	0	0	0	0	0	0	0	0
Education	0	1	0	0	0	0	0	0	0	0
Entertainment	0	0	4	0	0	0	0	0	0	0
Environment	0	0	0	6	0	0	0	0	0	0
Finance	0	0	0	0	3	0	0	0	0	0
Health	4	0	0	0	0	5	0	0	0	0
Politics	0	0	0	0	0	0	3	0	0	0
Sports	0	0	0	0	0	0	0	6	0	0
Technology	0	5	2	0	3	1	3	0	6	0
Travel	0	0	0	0	0	0	0	0	0	6

```
>
```

For hyperparameter tuning using 5-fold cross-validation. The `tune_grid()` iteratively goes through `mtry` and `min_n` leads to calculations of accuracy and Kappa statistic and `select_best()` is used to extract the comparatively accurate hyperparameter combination. The `conf_mat()` function generates a **confusion matrix**.

For trees=100

Output

The predictions are made on the test set using the trained model. The predictions are evaluated using performance metrics: reported **accuracy** of **70%** and a **Kappa score of 0.667**, indicating decent agreement. The `conf_mat()` function generates a **confusion matrix**, displays the model performance by the number of correct and incorrect predictions for each class. This helps identify which classes are misclassified.

The first row shows **Culture was predicted 2 times**, and both of those were correctly predicted as Culture.

The third row shows **Entertainment was predicted 4 times**

Similarly, for **Technology** it was predicted **6 times**.

2. How to improve prediction

For trees=500, mtry 5,20 and min_n 2-10

```
)  
) # 3. Specify a Random Forest Model with Tunable Hyperparameters  
# We'll tune mtry (number of predictors sampled for splitting)  
# and min_n (minimum number of observations in a node).  
) rf_spec <- rand_forest(  
  trees = 500, # We'll keep trees fixed for this tuning example# more trees more stable, hierarchical clust  
  mtry = tune(), # Number of predictors to sample at each split#set_engine("ranger")%>%pipe good for tree based models  
  min_n = tune() # Minimum number of data points in a nodeset  
) %>%  
) set_engine("ranger") %>%  
) set_mode("classification")#set_engine("ranger")%>%pipe good for tree based models  
)
```

```
>  
> print(final_metrics)  
# A tibble: 2 x 3  
  .metric .estimator .estimate  
  <chr>   <chr>      <dbl>  
1 accuracy multiclass 0.733  
2 kap      multiclass 0.704  
> # Confusion Matrix  
> final_conf_mat <- conf_mat(final_preds, truth = label, estimate = .pred_class)  
> print(final_conf_mat)  
      Truth  
Prediction Culture Education Entertainment Environment Finance Health Politics Sports Technology Travel  
Culture      2          0          0          0          0          0          0          0          0          0  
Education    0          6          2          0          3          1          3          0          3          0  
Entertainment 0          0          4          0          0          0          0          0          0          0  
Environment  0          0          0          6          0          0          0          0          0          0  
Finance      0          0          0          0          3          0          0          0          0          0  
Health       4          0          0          0          0          5          0          0          0          0  
Politics     0          0          0          0          0          0          3          0          0          0  
Sports       0          0          0          0          0          0          0          6          0          0  
Technology   0          0          0          0          0          0          0          0          3          0  
Travel       0          0          0          0          0          0          0          0          0          6  
>
```

The accuracy is 0.733 and the Kappa statistic is 0.704. A rise in Kappa 0.66 to 0.70 here.

The confusion matrix shows a solid diagonal pattern especially in categories like Sports, Health, and Travel indicating good performance compared to the previous one, also **Kappa of 0.704** values suggests substantial agreement beyond chance, adding reliability of model.

For tree = 500 and Changing mtry to 5, 30 and min_n to 2,12

```
# 5. Set Up Cross-Validation
set.seed(123)
cv_folds <- vfold_cv(train_data, v = 5, strata = label)

# 6. Define a Grid for Hyperparameter Tuning
# Here, we specify a grid for mtry and min_n.
rf_grid <- grid_regular(
  mtry(range = c(5, 30)),
  min_n(range = c(2, 12)),
  levels = 5 # 5 levels for each hyperparameter
)

> print(final_metrics)
# A tibble: 2 x 3
  .metric .estimator .estimate
<chr>    <chr>      <dbl>
1 accuracy multiclass 0.733
2 kap      multiclass 0.704
>
> # Confusion Matrix
> final_conf_mat <- conf_mat(final_preds, truth = label, estimate = .pred_class)
> print(final_conf_mat)
      Truth
Prediction Culture Education Entertainment Environment Finance Health Politics Sports Technology Travel
Culture      2          0          0          0          0          0          0          0          0          0
Education    0          6          2          0          0          3          1          3          3          0
Entertainment 0          0          4          0          0          0          0          0          0          0
Environment  0          0          0          6          0          0          0          0          0          0
Finance      0          0          0          0          3          0          0          0          0          0
Health       4          0          0          0          0          5          0          0          0          0
Politics     0          0          0          0          0          0          3          0          0          0
Sports       0          0          0          0          0          0          0          6          0          0
Technology   0          0          0          0          0          0          0          0          3          0
Travel       0          0          0          0          0          0          0          0          0          6
>
> # 10. Predict on New Samples (Optional)
> new_samples <- tibble(
+   text = c("The international film festival showcased diverse movies.",
+           "Renewable energy projects are being launched globally.",
+           "Financial markets are showing unusual volatility today.")
+ )
> new_preds <- predict(final_fit, new_data = new_samples)
> new_samples <- new_samples %>% bind_cols(new_preds)
> print(new_samples) # Note the misclassified cases
# A tibble: 3 x 2
  text .pred_class
<chr> <fct>
1 The international film festival showcased diverse movies. Sports
2 Renewable energy projects are being launched globally. Environment
3 Financial markets are showing unusual volatility today. Education
```

The output had no change.

For tree = 500 and Changing mtry to 5, 50 and min_n to 2,12

```
50
51 # 6. Define a Grid for Hyperparameter Tuning
52 # Here, we specify a grid for mtry and min_n.
53 rf_grid <- grid_regular(
54   mtry(range = c(5, 50)),
55   min_n(range = c(2, 12)),
56   levels = 5 # 5 levels for each hyperparameter
57 )
58
```

```
> # Performance Metrics
> final_preds <- final_preds %>% mutate(label = as.factor(label))
> final_metrics <- metric_set(accuracy, kap)(final_preds, truth = label, estimate = .pred_class)
>
> print(final_metrics)
# A tibble: 2 x 3
  .metric .estimator .estimate
  <chr>   <chr>      <dbl>
1 accuracy multiclass 0.767
2 kap      multiclass 0.741
>
> # Confusion Matrix
> final_conf_mat <- conf_mat(final_preds, truth = label, estimate = .pred_class)
> print(final_conf_mat)
```

Prediction \ Truth	Culture	Education	Entertainment	Environment	Finance	Health	Politics	Sports	Technology	Travel
Culture	6	0	0	0	0	0	0	0	0	0
Education	0	1	0	0	0	0	0	0	0	0
Entertainment	0	0	4	0	0	0	0	0	0	0
Environment	0	0	0	6	0	0	0	0	0	0
Finance	0	0	0	0	3	0	0	0	0	0
Health	0	0	0	0	0	5	0	0	0	0
Politics	0	0	0	0	0	0	3	0	0	0
Sports	0	0	0	0	0	0	0	6	0	0
Technology	0	5	2	0	3	1	3	0	6	0
Travel	0	0	0	0	0	0	0	0	0	6

Here, the model achieved **higher accuracy (0.767)** and **Kappa (0.741)** compared to the second screenshot's **accuracy (0.733)** and **Kappa (0.704)**, indicating improved overall performance and agreement beyond chance. This improvement is also evident in the **confusion matrix**, with zero misclassifications across all 10 categories.

Overall Conclusion

- **Increasing trees from 100 to 500 improved accuracy and stability** of predictions.
- **Expanding mtry from 5–20 to 5–30 gives notable gain** the model with mtry 5–30 and min_n 2–12 reached **0.767 accuracy and class-wise predictions**.

b. Add your own name and publish on your website